

IA com Ferramentas Externas

Quando trabalhamos com Large Language Models (LLMs) em sua forma nativa, isto é, apenas com o modelo de pesos pré-treinados, lidamos com uma espécie de "caixa fechada". Um LLM reúne muito conhecimento sobre o mundo até sua data de corte de treinamento, mas não enxerga o presente, não interage diretamente com sistemas, não acessa a internet em tempo real e, sobretudo, enfrenta grandes dificuldades com cálculos determinísticos ou raciocínios exatos de longo escopo.

A principal limitação dos modelos generativos isolados é a **incapacidade de executar ações**. Eles geram texto de forma probabilística. Por isso, diante de uma solicitação sobre o clima atual em Palmas, um ChatGPT puramente textual, sem conexão com ferramenta externa, tende a alucinar uma resposta ou a avisar que não possui essa capacidade.

Uma mudança relevante na engenharia de sistemas de IA recentes não está apenas na melhoria dos modelos de texto, mas na possibilidade de usá-los como **motores de raciocínio (Reasoning Engines)** capazes de delegar ações a **ferramentas externas**.

Geração de Texto vs Execução de Ação no Mundo Real

É importante diferenciar a **Geração** da **Execução**:

- **Geração:** O modelo prevê a próxima palavra. Ele "escreve" uma resposta com base em padrões aprendidos. Exemplo: "O clima em Palmas geralmente é muito quente em agosto."
- **Execução:** Um código em um servidor local ou na nuvem faz uma requisição HTTP, aciona um banco de dados, escreve um arquivo, etc. Exemplo:
`GET https://api.open-meteo.com/v1/forecast?latitude=-10.1675&longitude=-48.3277¤t=temperature_2m.`

Para unir essas duas dimensões, usamos **Tool Calling** (ou Function Calling). Nesse paradigma, o LLM não executa a ação diretamente. Ele emite uma intenção de execução em formato estruturado, geralmente JSON. Em seguida, a nossa aplicação, que funciona como orquestradora, recebe esse JSON, executa o código Python/Node correspondente e devolve o resultado ao LLM.

Contratos Claros: O Schema da Ferramenta

Para que a IA saiba quais ferramentas existem e como deve usá-las, o desenvolvedor precisa definir um **contrato claro**. Esse contrato geralmente usa o formato JSON Schema e inclui:

1. **Nome da Ferramenta:** Ex: `consultar_clima`.
2. **Descrição:** Ex: "Obtém a temperatura atual e as condições climáticas de uma cidade específica." A descrição é a parte central do contrato, pois é ela que o LLM lê para decidir se deve usar a ferramenta.

3. **Parâmetros:** Ex: cidade (string, obrigatório), unidade (string, opcional, enum: ['Celsius', 'Fahrenheit']).

Se o contrato for mal escrito, o LLM poderá enviar parâmetros incorretos, como "Brasil" no lugar de uma cidade, ou invocar a ferramenta no momento errado.

O Fluxo de Execução

Em vez do fluxo tradicional de solicitação e resposta, o uso de ferramentas funciona como um ciclo supervisionado. A diferença principal é que o LLM deixa de ser o único responsável pela resposta final. Ele passa a atuar como uma camada de interpretação: entende o pedido do usuário, decide se precisa de uma ferramenta e informa ao sistema qual ação deve ser executada.

Em uma solicitação sobre a temperatura atual em Palmas, uma resposta baseada apenas no treinamento poderia soar plausível, mas estaria desatualizada ou inventada. Com Tool Calling, o comportamento esperado é outro. O LLM identifica que a solicitação depende de um dado externo e atual. Em vez de responder diretamente ao usuário, ele gera uma chamada estruturada, como:

```
{"tool": "consultar_clima", "args": {"cidade": "Palmas"}}
```

Esse JSON não é a resposta final. Ele é uma instrução para o aplicativo hospedeiro, também chamado de orquestrador. O orquestrador lê a chamada, verifica se a ferramenta existe, valida os argumentos recebidos e executa o código real, por exemplo: `resultado = weather_api.get("palmas")`. Nesse momento, quem acessa a API, trata autenticação, lida com erro de rede e recebe os dados concretos é a aplicação, não o modelo.

Depois da execução, o resultado volta para o LLM como uma nova informação de contexto: "A ferramenta retornou: {temperatura: 32, condicao: Ensolarado}". Só então o modelo produz a resposta em linguagem natural: "A temperatura em Palmas é de 32 graus e o clima está ensolarado."

Esse fluxo ensina uma ideia importante de arquitetura: o LLM decide e sintetiza, mas não deve ser tratado como a fonte de verdade para dados externos. A fonte de verdade é a ferramenta acionada pelo sistema. Por isso, uma implementação confiável precisa separar bem três responsabilidades:

1. **O modelo interpreta a intenção:** ele identifica que a solicitação exige uma consulta externa.
2. **O orquestrador controla a execução:** ele valida a chamada, executa a função correta e impede ações inesperadas.
3. **A ferramenta produz o dado verificável:** ela consulta a API, o banco de dados ou o serviço necessário e devolve um resultado objetivo.

Na prática, essa separação reduz alucinações e torna o sistema mais auditável. Se algo der errado, o desenvolvedor consegue investigar em qual etapa o problema ocorreu: escolha incorreta de ferramenta, argumentos mal formatados, erro de API ou falha do orquestrador ao bloquear uma chamada inválida.

Pensar nesse fluxo como uma cadeia de responsabilidades ajuda a projetar aplicações de IA mais seguras, testáveis e úteis.

Controle de Fluxo e Validação de Retorno

Como arquitetos do sistema, somos nós que decidimos **quando usar a IA** e **quando acionar a ferramenta**. Em alguns casos, podemos forçar o LLM a sempre responder por meio de uma ferramenta, o que é útil em tarefas de extração de dados.

Um ponto crítico é a **validação de retorno**. Quando a API do clima está fora do ar, a ferramenta deve retornar ao LLM uma mensagem como "Erro: API indisponível". O modelo, então, pode ler o erro e informar o usuário de forma fluida: "Desculpe, no momento não consigo consultar o clima porque o serviço está fora do ar".